大家好，我是 Guide。

最近我把项目里「文件上传解析与 PDF 导出」这一块做了一次系统复盘，顺手把涉及到的高频面试题也整理成了第一版：从**文档解析、文本清洗、去重、异步任务**到**PDF 生成、中文字体、接口设计与性能优化**，尽量把“为什么这么做”和“怎么落地”讲透。

坦白说，文件解析/导出看起来是“工程细节”，但一旦做成线上能力，它就是典型的**坑多、边界多、稳定性要求极高**的模块：格式千奇百怪、解析噪音污染严重、恶意文件与 OOM 风险并存；同时 AI 分析和大报告导出又天然耗时，稍不注意就会把同步接口拖死，影响整体吞吐。既可以把它当作面试题速查，也可以当作一份可直接复用的工程落地清单。

文件解析

☆ 为什么选择 Apache Tika 作为文档解析库？

Apache Tika 是一个成熟的开源"内容分析工具箱"，能从上千种不同格式的文件中，通过统一接口提取出文本和元数据。

技术选型对比：

方案	格式支持	核心优势	局限性
Apache Tika	极高	统一 API；自动识别类型；可集成 OCR；社区生态强	依赖包偏大；默认策略会带来噪音
Apache POI	中 (Office)	对 Word/Excel 的结构控制细（段落、表格、样式）	不支持 PDF；多格式需拼装
PDFBox	低 (PDF)	PDF 解析最稳定；支持坐标定位	只覆盖 PDF；扫描件需 OCR
Pandoc	极高	格式转换的"瑞士军刀"，排版还原度高	需系统级安装（非纯 Java）；并发性能差

选择 Tika 的理由：

1. **统一 API**：无论是 PDF、Word、Excel 还是其他格式，都通过同一套接口处理，降低开发复杂度
2. **自动类型识别**：基于魔数/内容判断 MIME 类型，不依赖文件后缀，更安全可靠
3. **可扩展性强**：底层封装了 PDFBox、POI 等专业库，既享受统一接口的便利，又保留了底层能力

☆ Tika 默认解析方式有什么问题？你是如何优化的？

> 类似的问题：为什么不推荐 `new Tika().parseToString()`？它有哪些坑？

默认方式的问题：

```
// ❌ 不推荐：简单但有问题
Tika tika = new Tika();
String text = tika.parseToString(inputStream);
```

问题一：嵌入资源污染

Word 文档中的图片会被提取为文件名引用：

```
姓名：张三
image1.jpeg            ← 简历中的头像图片
工作经验：5年Java开发
```

问题二：PDF 临时路径泄露

PDF 中的内嵌资源会产生临时文件路径：

```
技术栈：Java, Spring Boot
file:///tmp/apache-tika-123456.html?query=0     ← 临时文件路径
```

问题三：无长度限制

恶意文件可能产生超大文本，导致 OOM。

优化方案：显式 Parser + Context

```java
private String parseContent(InputStream inputStream) {
    AutoDetectParser parser = new AutoDetectParser();

    // 限制最大文本长度，防止 OOM
    BodyContentHandler handler = new BodyContentHandler(MAX_TEXT_LENGTH);
    Metadata metadata = new Metadata();
    ParseContext context = new ParseContext();

    // 关键配置 1：将 Parser 注册到 Context（支持递归解析）
    context.set(Parser.class, parser);

    // 关键配置 2：禁用嵌入文档提取（解决图片污染）
    context.set(EmbeddedDocumentExtractor.class,
        new NoOpEmbeddedDocumentExtractor());

    // 关键配置 3：PDF 专用配置
    PDFParserConfig pdfConfig = new PDFParserConfig();
    pdfConfig.setExtractInlineImages(false);
    context.set(PDFParserConfig.class, pdfConfig);

    parser.parse(inputStream, handler, metadata, context);
    return textCleaningService.cleanText(handler.toString());
}
```

☆你们是怎么禁用嵌入资源提取的？

`NoOpEmbeddedDocumentExtractor`是解决嵌入资源污染的核心类，它实现了`EmbeddedDocumentExtractor`接口，重写`shouldParseEmbedded()`直接返回`false`，并通过`ParseContext`注入：

```java
public class NoOpEmbeddedDocumentExtractor implements EmbeddedDocumentExtractor
{

    @Override
    public boolean shouldParseEmbedded(Metadata metadata) {
        // 返回 false，跳过所有嵌入资源
        return false;
    }
```

```
    @Override
    public void parseEmbedded(InputStream stream, ContentHandler handler,
            Metadata metadata, boolean outputHtml) {
        // 空实现，不执行任何操作
    }
}
```

工作原理：

Tika 在解析过程中会调用 `EmbeddedDocumentExtractor` 接口：

1. `shouldParseEmbedded()` - 询问是否要处理嵌入资源（返回 false 直接跳过）
2. `parseEmbedded()` - 实际处理嵌入资源（因上一步返回 false 不会被调用）

哪些资源会被跳过?

文档类型	嵌入资源示例	默认行为	使用 NoOp 后
Word (DOCX)	图片、图表、OLE 对象	输出 `image1.jpeg`	跳过
PDF	内嵌图片、附件	输出临时路径	跳过
Excel	嵌入图表、图片	输出引用	跳过

为什么需要双层清理策略?

Tika 负责"解析"，`TextCleaningService` 负责"清洗"。两者分工明确，形成多层防御：

场景	Tika 处理	TextCleaningService 处理
Word 嵌入图片	☑ NoOp 跳过	兜底清理
PDF 临时路径	部分场景仍会泄露	☑ 正则过滤
特殊符号分隔线	✕ 无法处理	☑ 正则过滤
连续空行	✕ 无法处理	☑ 格式压缩
控制字符	✕ 无法处理	☑ 正则过滤

TextCleaningService 核心清理规则：

```
// 语义去噪
t = CONTROL_CHARS.matcher(t).replaceAll("");       // 控制字符
t = IMAGE_FILENAME_LINE.matcher(t).replaceAll(""); // image1.jpeg
t = IMAGE_URL.matcher(t).replaceAll("");            // HTTP图片链接
t = FILE_URL.matcher(t).replaceAll("");             // file:///tmp/...
t = SEPARATOR_LINE.matcher(t).replaceAll("");       // ---, ===

// 格式规范化
t = t.replace("\r\n", "\n").replace("\r", "\n");    // 统一换行符
t = t.replaceAll("\\n{3,}", "\n\n");                // 压缩空行
```

文件上传的完整流程是怎样的?

八步上传流程：

1. **文件验证**：非空、大小限制（如 10MB）、业务类型提示（简历）。
2. **类型识别**：用解析服务检测 MIME（不信后缀），拒绝不支持类型。
3. **去重**：基于 **SHA-256** 的内容哈希查重，命中则返回历史分析结果。
4. **文本解析**：Tika 解析 + 二次清洗，保证进入 AI 的文本质量。
5. **对象存储**：上传到 RustFS/S3，拿到 `fileKey` / `fileUrl`。
6. **落库**：保存 Resume 记录，状态置为 `PENDING`。
7. **异步分析**：发送 Redis Stream 任务（避免同步 5~30 秒阻塞）。
8. **返回结果**：同步返回 ID、状态、下载链接等。

文件去重是如何实现的？

采用基于内容的 SHA-256 哈希去重：

```
public String calculateFileHash(MultipartFile file) {
    MessageDigest digest = MessageDigest.getInstance("SHA-256");
    byte[] hash = digest.digest(file.getBytes());
    return bytesToHex(hash);
}

// 上传时检查
Optional<ResumeEntity> existingResume = persistenceService.findByFileHash(hash);
if (existingResume.isPresent()) {
    return handleDuplicateResume(existingResume.get());
}
```

为什么用 SHA-256 而不是 MD5？

- MD5 已被证明存在碰撞漏洞，不再推荐用于安全场景
- SHA-256 碰撞概率极低（2^128），更安全可靠
- 对于文件去重场景，SHA-256 的计算开销是可接受的

为什么文件分析要异步处理？

AI 分析简历通常需要 **5-30 秒**，同步处理会导致：

- 用户等待时间过长，体验差
- HTTP 连接超时（默认 30s）
- 服务器线程阻塞，影响并发

解决方案：Redis Stream 异步处理。

详细介绍可以查看这篇文章：[☆基于 Redis Stream 的异步任务处理实现]。

用户上传空文件如何处理？

分层校验策略：

1. **前端校验**：文件大小小于某阈值（如 10KB 的 PDF）直接拦截
2. **后端解析**：`DocumentParseService` 检测到空文件返回空字符串
3. **业务层校验**：检测到空内容抛出友好异常

```
String content = parseService.parseContent(file);
if (content == null || content.trim().isEmpty()) {
    throw new BusinessException(ErrorCode.INTERNAL_ERROR,
        "无法从文件中提取文本内容，请确保文件格式正确");
}
```

这种分层校验不仅节省了 AI 调用成本，也确保了系统能给用户提供精准的反馈。

PDF 导出

☆ 为什么选择 iText 8 作为 PDF 生成库?

技术选型对比:

对比项	iText 8	OpenPDF	Apache PDFBox
许可证	AGPL（商用需付费）	LGPL/MPL（免费商用）	Apache 2.0（免费商用）
功能丰富度	☆☆☆☆☆	☆☆☆	☆☆☆☆
API 易用性	☆☆☆☆	☆☆☆☆☆	☆☆☆
文档布局能力	强大（表格、多列、样式）	基础	底层控制
中文支持	优秀（font-asian 模块）	良好	一般

选择 iText 8 的理由:

1. **功能完备性**：原生支持复杂的文档布局需求，包括多级表格嵌套、灵活的段落样式控制、精确的页面排版
2. **中文排版支持**：通过 `font-asian` 模块，对中日韩文字提供开箱即用的支持
3. **成熟度与稳定性**：经过二十余年迭代打磨，API 设计成熟，文档齐全，社区活跃
4. **企业级特性**：支持 PDF/A 归档标准、数字签名、表单处理等

注意：iText 8 社区版采用 AGPL 许可证，闭源商业项目需要评估是否符合条款或购买商业许可。

☆ PDF 中文显示乱码如何解决?

PDF 中文显示是一个常见痛点。项目采用**内嵌字体方案**，确保跨平台一致性:

```
private PdfFont createChineseFont() {
    // 使用项目内嵌字体（保证跨平台一致性）
    var fontStream = getClass().getClassLoader()
        .getResourceAsStream("fonts/ZhuqueFangsong-Regular.ttf");
    if (fontStream != null) {
        byte[] fontBytes = fontStream.readAllBytes();
        return PdfFontFactory.createFont(
            fontBytes,
            PdfEncodings.IDENTITY_H,            // 支持 Unicode
            EmbeddingStrategy.FORCE_EMBEDDED    // 强制嵌入字体到 PDF
        );
    }
    throw new BusinessException(ErrorCode.EXPORT_PDF_FAILED, "字体文件缺失");
}
```

关键点:

- 字体文件放在 `src/main/resources/fonts/` 目录
- 使用 `PdfEncodings.IDENTITY_H` 支持 Unicode
- `EmbeddingStrategy.FORCE_EMBEDDED` 强制嵌入字体到 PDF

为什么不用系统字体?

系统字体检测和降级逻辑复杂且容易出错。不同操作系统的字体路径不同（Windows: `C:\Windows\Fonts`，macOS: `/Library/Fonts`），可能导致:

- 字体找不到
- 不同系统显示效果不一致
- 部分系统缺少中文字体导致乱码

PDF 导出时特殊字符如何处理?

为避免 emoji 等特殊字符导致渲染问题:

```
private String sanitizeText(String text) {
    if (text == null) return "";
    // 移除可能导致问题的特殊字符（如 emoji）
    return text.replaceAll("[\\p{So}\\p{Cs}]", "").trim();
}
```

- `\p{So}`: 其他符号（包括 emoji）
- `\p{Cs}`: 代理对字符（用于表示超出基本多文种平面的字符）

PDF 导出接口如何设计?

```
@GetMapping("/api/interview/sessions/{sessionId}/export")
public ResponseEntity<byte[]> exportInterviewPdf(@PathVariable String sessionId)
{
    byte[] pdfBytes = historyService.exportInterviewPdf(sessionId);
    String filename = URLEncoder.encode(
        "模拟面试报告_" + sessionId + ".pdf",
        StandardCharsets.UTF_8
    );

    return ResponseEntity.ok()
        .header(HttpHeaders.CONTENT_DISPOSITION,
            "attachment; filename*=UTF-8''" + filename)
        .contentType(MediaType.APPLICATION_PDF)
        .body(pdfBytes);
}
```

注意点:

- 使用 `filename*=UTF-8''` 格式支持中文文件名（RFC 5987）
- 返回 `application/pdf` MIME 类型
- 设置 `attachment` 触发浏览器下载而非预览

如果 PDF 生成耗时很长怎么办?

如果报告内容非常多（比如包含大量的 RAG 检索依据），生成 PDF 可能会超过 5 秒。建议补充说明: 可以利用项目中已有的 **Redis Stream**，将 PDF 生成任务异步化，生成完成后存储在 S3 (MinIO)，再通过消息/轮询通知用户下载。

有没有更优雅的 PDF 生成方式?

手动编写 `document.add(new Paragraph(...))` 代码非常繁琐且难以维护。

推荐方案：Thymeleaf + pdfHTML

iText 8 的扩展模块 pdfHTML 能将 HTML/CSS 内容直接转换为 PDF：

```
// 1. 使用 Thymeleaf 渲染 HTML 模板
Context context = new Context();
context.setVariable("report", reportData);
String html = templateEngine.process("report-template", context);

// 2. HTML 转 PDF
HtmlConverter.convertToPdf(html, new FileOutputStream("report.pdf"));
```

优势：

- **关注点分离**：HTML 负责结构，CSS 负责样式，Java 只负责数据准备
- **快速迭代**：调整样式只需修改 HTML/CSS，无需重新编译
- **复用前端技能**：用熟悉的 HTML/CSS 设计 PDF
- **易于维护**：模板文件比 Java 代码更直观

综合问题

文件上传有哪些安全风险？如何防范？

风险	防范措施
文件类型伪造	基于内容（魔数）检测类型，不依赖后缀
恶意文件上传	白名单限制允许的 MIME 类型
文件过大导致 OOM	限制文件大小（如 10MB）
路径注入	清理文件名特殊字符，使用 UUID 重命名
病毒文件	集成杀毒软件扫描（ClamAV）

```
// 类型白名单校验
private static final Set<String> ALLOWED_TYPES = Set.of(
    "application/pdf",
    "application/msword",
    "application/vnd.openxmlformats-officedocument.wordprocessingml.document"
);

public void validateContentType(String contentType) {
    if (!ALLOWED_TYPES.contains(contentType)) {
        throw new BusinessException(ErrorCode.INVALID_FILE_TYPE);
    }
}
```

大文件上传如何优化？

1. **分片上传**：将大文件切分为多个小块并行上传，支持断点续传。
2. **流式处理**：使用 `InputStream` 而非 `getBytes()`，避免内存溢出。
3. **异步处理**：上传和解析分离，快速响应用户。
4. **进度反馈**：WebSocket 推送上传进度。

如何保证文件存储的可靠性？

1. **对象存储**：使用 S3 兼容存储（MinIO/OSS），天然支持高可用。
2. **元数据分离**：文件内容存对象存储，元数据存数据库。
3. **一致性保证**：先写对象存储成功后再写数据库，失败时补偿删除。
4. **定期清理**：定时任务清理孤儿文件（数据库无记录但存储存在的文件）。